Харківський національний університет імені В.Н. Каразіна

Кафедра інтелектуальних програмних систем і технологій

Дисципліна: Методи контейнерізації та віртуалізації програмних рішень

ЛАБОРАТОРНА РОБОТА №4

Створення Dockerfile. Опанування утіліти docker compose. Знайомство з Docker Hub.

Харків 2025 рік

Зміст

Опис 3
Мета лабораторної роботи 3
Схема оцінювання 3
Завдання 4
Завдання №1 [I]. Розробка Dockerfile для сервісу 5
Приклад на Python 5
Приклад на .Net 5
Приклад на Java 5
Завдання №2 [I]. Конфігурація Container Registry в Docker Desktop 6
Завдання №3 [K]. Розробка docker compose файлу для запуску всіх сервісів проекту 7
Завдання №4 [K]. Інтеграція бази даних в docker compose файл 8
Додаток А. Приклад на Python 9
Dockerfile 9
Список додаткових джерел інформації 10
Додаток Б. Приклад на C# 11
Dockerfile 11
Список додаткових джерел інформації 11
Додаток В. Приклад на Java 12
Dockerfile 12
Список додаткових джерел інформації 12
Додаток Г. Налаштування доступу к Container Registry в Docker Desktop. 13
Реєстрація 13
Створення PAT (personal access token) 15
Створення Container Registry репозіторія 17
Робота з Container Registry 17
Додаток Д. Утіліта docker-compose 20

Опис

Мета лабораторної роботи

1. Отримання практичних навичок в розробці web-сервісів.
2. Вдосконалення навичок роботи з дистрибутивною системою контролю версій – Git.
3. Отримання знань щодо Docker
4. Опанування інструменту Docker compose.
5. Планування подальшої роботи.

Схема оцінювання

Завдання	Критерії прийняття	Бал
Завдання №1 [I]. Розробка Dockerfile для сервісу.	• Продемонструвати створення Docker Image с заданою версією • Продемонструвати працездатність сервісу • Продемонструвати, що переданий аргумент був використован конфігурацією	1
Завдання №2 [I]. Конфігурація Container Registry в Docker Desktop	• Продемонструвати наявність Docker image в Docker Hub репозіторії	1
Завдання №3 [K]. Розробка docker compose файлу для запуску всіх сервісів проекту	• Продемонструвати, що всі сервіси проекту були запущені за допомогою docker compose	1
Завдання №3 [K]. Інтеграція бази даних в docker compose файл	• Запустити контейнер с базою даних та продемонсрувати можливість доєднання до неї за допомогою утіліти DBeaver	2

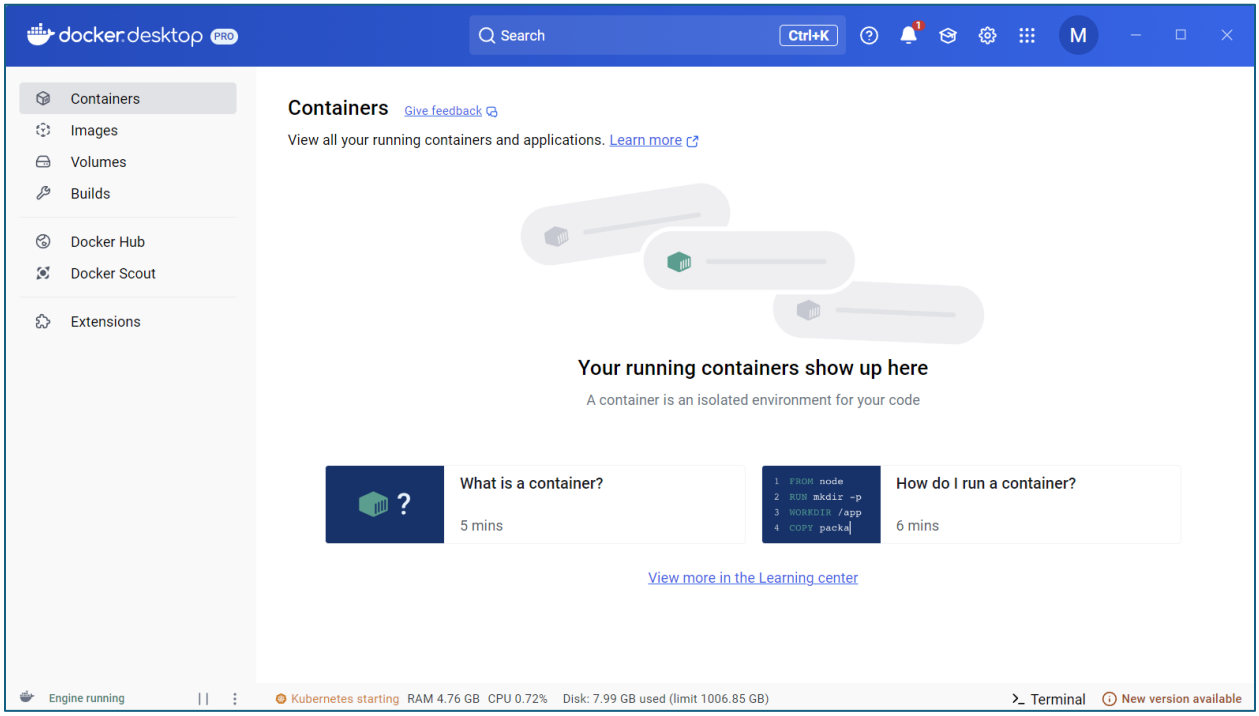
[K] – колективне завдання, оцінку отримує вся команда в незалежності від того, хто скільки зусиль доклав. [I] – інивідуальне завдання, оцінку отримує конкретний виконавець.

Завдання

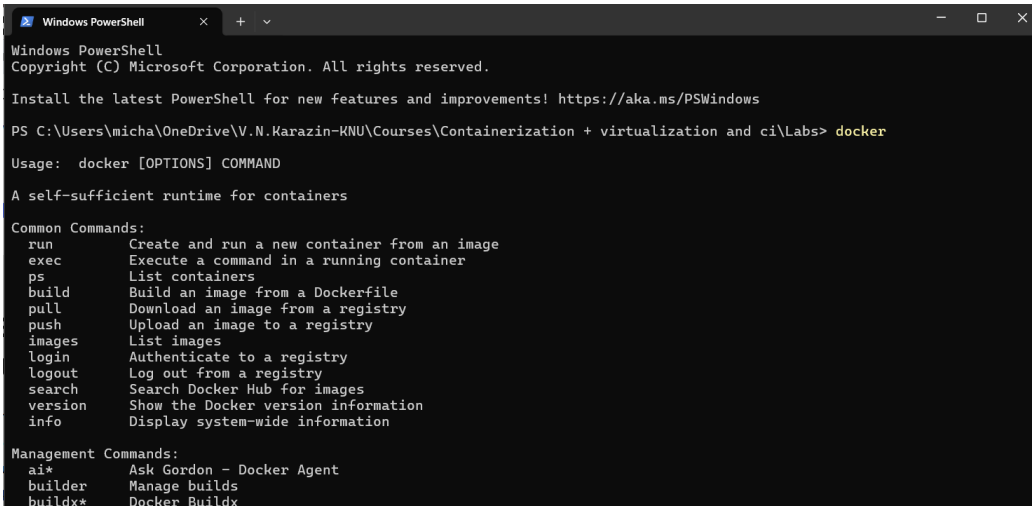
Перед тим як розпочати роботу з Docker, нам потрібно встановити відповідну програму на свою машину. Дуже популярними рішеннями є Docker Desktop та Rancher Desktop. Оберіть будь яку з них, зареєструйтесь та встановіть на вашу машину.

Далі в лабораторній я буду наводити скріншоти з системи Docker Desktop, але вона мало чим відрізняється від Rancher Desktop.

Після установки ви побачете наступний екран:



Також перевірте доступність Docker CLI (command line interface), запустивши в терміналі наступну команду: docker



Завдання №1 [I]. Розробка Dockerfile для сервісу

В контексті даної задачі, ми розробимо Dockerfile для нашого сервіса, навчимось працювти з командами Docker та протестуємо працездатність нашого контейнеру.

Далі ви знайдете вимоги до функціональності, а в додатку – інформацію про те як саме її можна реалізувати на обраній мові програмування.

№	Опис	Приклад	Коментар
Dockerfile			
A1	Додати в проект Dockerfile з описом необхідним для запуску сервіса в контексті контейнера.		
A2	Додати в Dockerfile передачу аргумента AppVersion, що повинен встановлювати відповідну зміну середовища та перезаписувати значення в конфігурації web-service.	Ендпоінт сервісу '/app/info', який був реалізован в попередній лабораторній роботі та який повертає інформацію про конфігурацію, повинен повертати значення що передано в контейнер.	

*Функціональну вимогу A2 можна виконати, якщо використовувати комбінацію ARG та ENV – ключові слова в контексті синтаксису Dockerfile (Build variables)

Приклад на Python

Детальний огляд на мові програмування Python ви можете знайти в Додаток А.

Приклад на .Net

Детальний огляд на мові програмування C# ви можете знайти в Додаток Б.

Приклад на Java

Детальний огляд на мові програмування Java з ви можете знайти в Додаток В.

Завдання №2 [I]. Конфігурація Container Registry в Docker Desktop

№	Опис	Приклад	Коментар
Створити Container Registry в Docker Hub а налаштувати доступ			
Б1	Створити акаунт в Docker Desktop		
Б2	Налаштувати доступ до Docker Hub – сховищу, що буде зберігати Docker Image та, в майбутньому, Helm Chart проекту.		
Б3	«Запушити» docker image web-сервісу на Azure DevOps Artifacts		

Детальну інформацію про налаштування можна знайти в Додаток Г. Налаштування доступу к Container Registry в Docker Desktop.

Завдання №3 [К]. Розробка docker compose файлу для запуску всіх сервісів проекту

В контексті даної задачі, ми розробимо docker-compose.yaml файл, що дозволить запустити всі сервіси, що відноситься до проекту.

№	Опис	Приклад	Коментар
docker-compose.yaml			
B1	Додати в проект docker-compose.yaml файл з описом необхідним для запуску сервісів. Продемонструвати запуск сервісів та їх працесдатність.		

Детальну інформацію, щодо утіліти docker compose можна знайти тут: Додаток Д. Утіліта docker-compose

Завдання №4 [К]. Інтеграція бази даних в docker compose файл

№	Опис	Приклад	Коментар
Додати інтеграцію бази даних в docker-compose.yaml			
Г1	Додати в docker-compose.yaml файл інтеграцію бази даних.		

Інтегрувати Docker Image з базою даних в docker-compose файл спробуйте самостійно. Офіціальний образ PosgreSql: https://hub.docker.com/_/postgres

Ви також можете обрати будь-яку іншу базу даних.

Для роботи з базою даних раджу встановити: https://dbeaver.io/

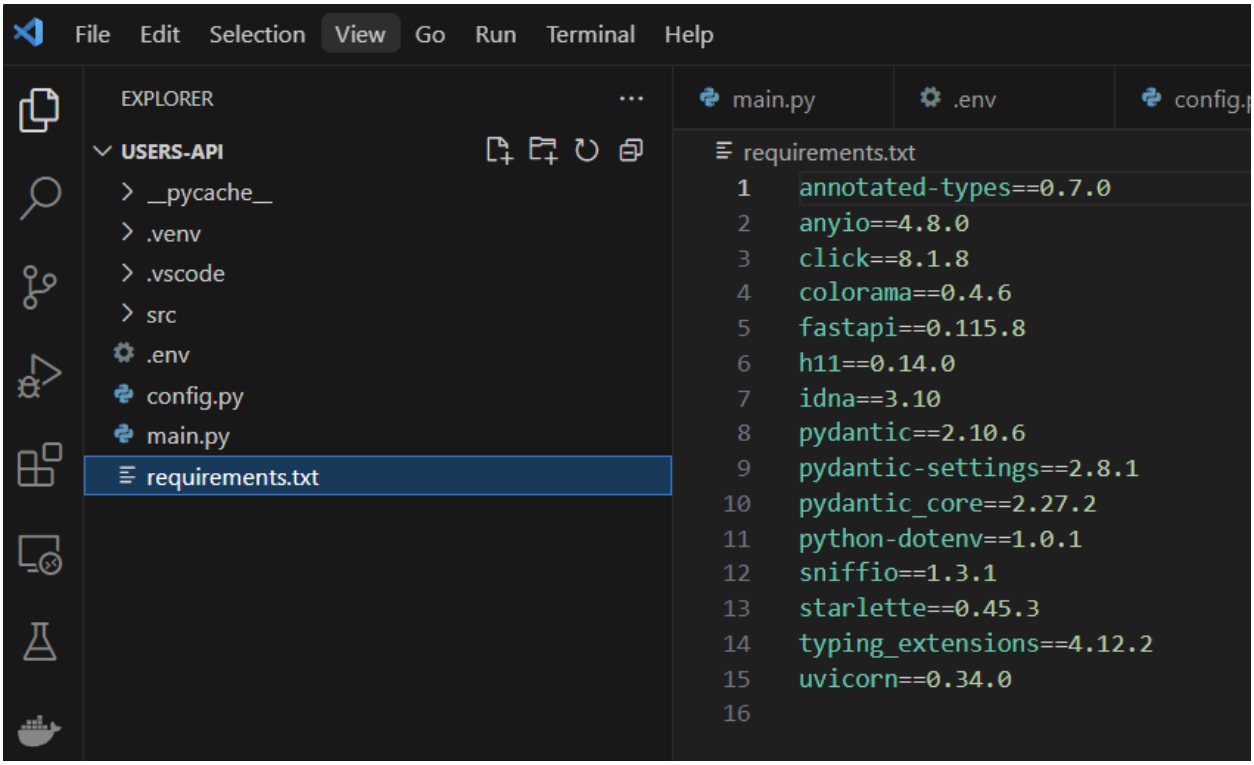
Додаток А. Приклад на Python

Dockerfile

Перед тим як, створити Dockerfile, необхідно сгенерувати файл с залежностями – requirements.txt. Для цього:

1. Активуйте віртуальне середовище:
   .\.venv\Scripts\Activate.ps1 (ящо на Mac, то activate.bat)
2. Виконайте команду:
   pip freeze > requirements.txt

Після цього у вас з'явиться файл requirements.txt з всіма залежностями.



Dockerfile для Python додатку може виглядати наступним чином:

```
# Use official Python image as base (slim version for reduced size)
FROM python:3.11-slim
# Set working directory
WORKDIR /app

# Copy only requirements first to leverage Docker caching
COPY ./requirements.txt .

# Install dependencies with pip
```

```
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

# Copy application files
COPY . .

# Expose FastAPI default port
EXPOSE 8000

# Command to run the FastAPI app with Uvicorn
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Список додаткових джерел інформації

1. Settings Management

Додаток Б. Приклад на C#

Dockerfile

Dockerfile для .Net/C# додатку може виглядати наступним чином:

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base
WORKDIR /app
EXPOSE 8080
EXPOSE 443

FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY ["src/", "./"]
RUN dotnet restore "/src/KHNU.Users.Api/KHNU.Users.Api.csproj"
COPY . .
WORKDIR "/src/KHNU.Users.Api"
RUN dotnet build "KHNU.Users.Api.csproj" -c Release -o /app/build

FROM build AS publish
RUN dotnet publish "KHNU.Users.Api.csproj" -c Release -o /app/publish

FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "KHNU.Users.Api.dll"]
```

Не забувайте, що шляхи описані в Dockerfile залежать від контексту, в якому запущена сборка.

Список додаткових джерел інформації

1. Overriding appsettings.json configuration

Додаток В. Приклад на Java

Dockerfile

Dockerfile для Spring Boot Java додатку може виглядати наступним чином:

```
# Use an official Maven image to build the application
FROM maven:3.9.6-eclipse-temurin-21 AS build

# Set the working directory
WORKDIR /app

# Copy the pom.xml and dependencies first to leverage Docker caching
COPY pom.xml ./
RUN mvn dependency:go-offline -B

# Copy the entire project and build it
COPY . ./
RUN mvn clean package -DskipTests

# Use a lightweight JDK runtime image for running the application
FROM openjdk:21-jdk-slim

# Set the working directory
WORKDIR /app

# Copy the built JAR file from the previous stage
COPY --from=build /app/target/*.jar app.jar

# Expose application port
EXPOSE 8080

# Run the Spring Boot app
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Не забувайте, що шляхи описані в Dockerfile залежать від контексту, в якому запущена сборка.

Список додаткових джерел інформації

1. Java Spring Boot Overriding Environment Variables Guide

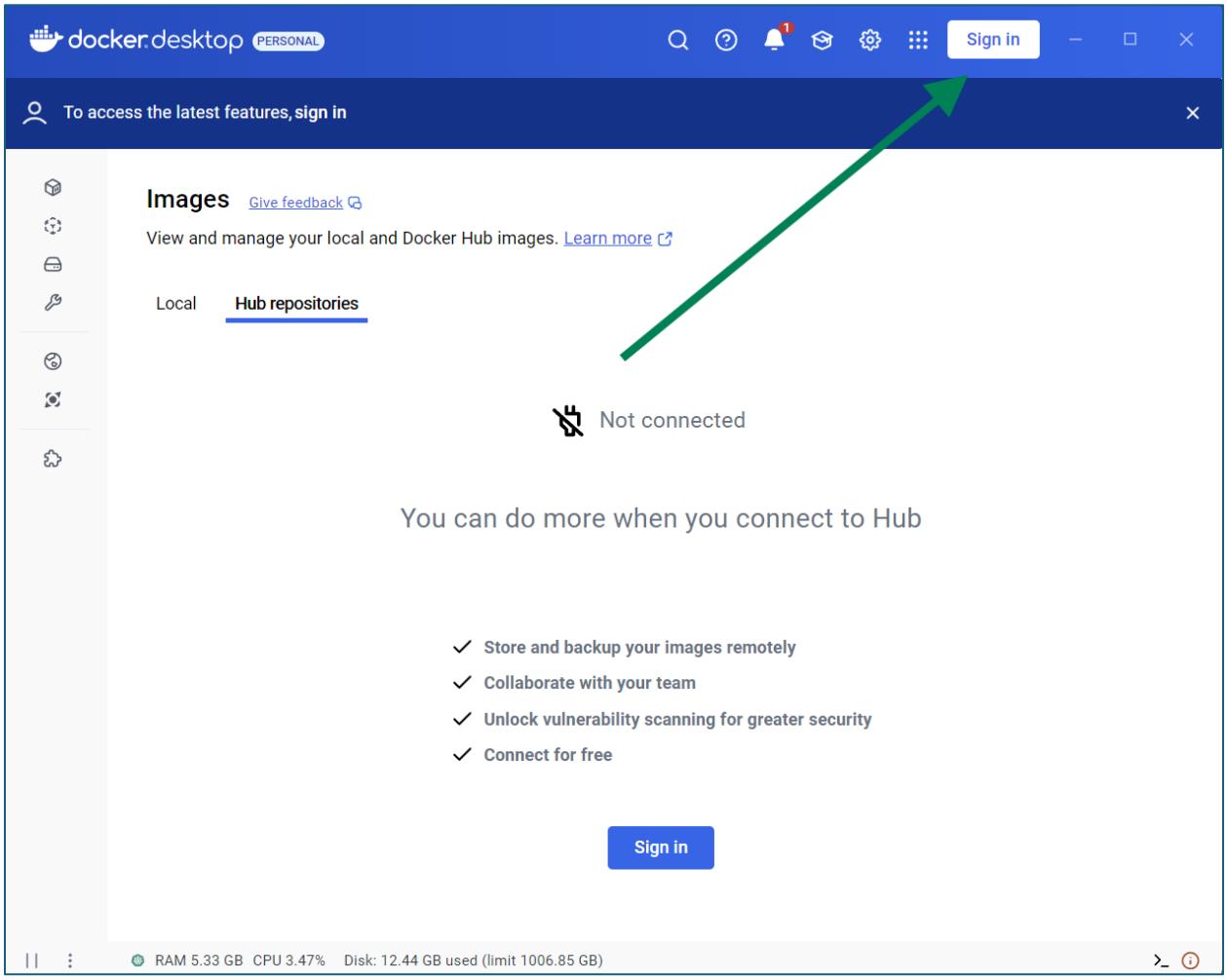
Додаток Г. Налаштування доступу к Container Registry в Docker Desktop.

Це загальна частина, яка не залежить від мови програмування.

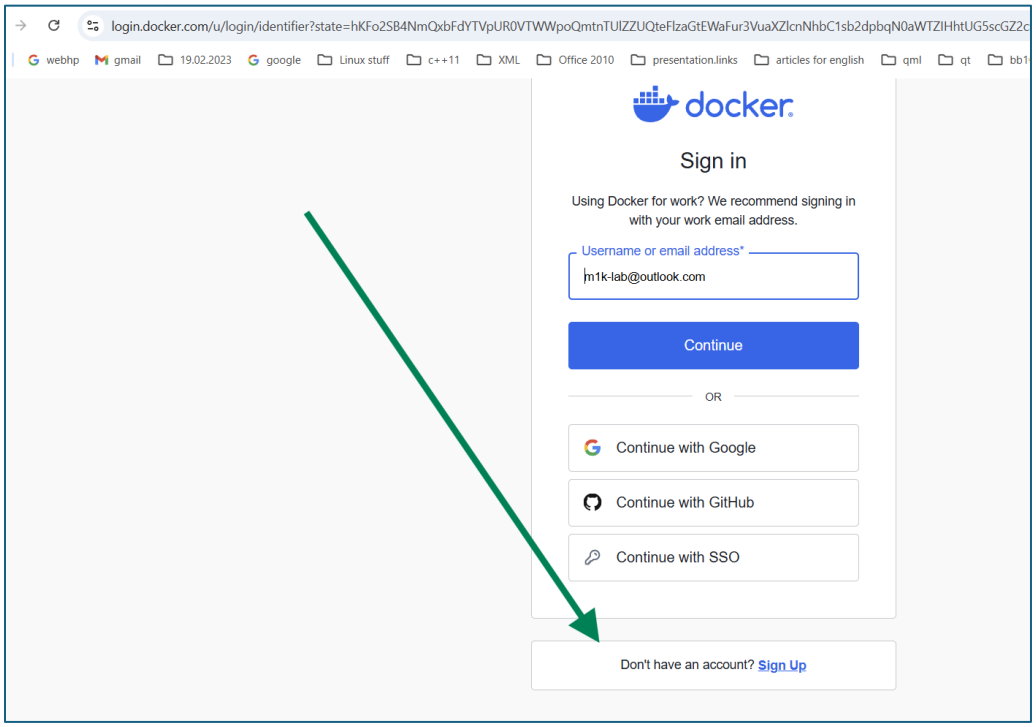
Реєстрація

Після того як ви встановити Docker Hub, у вас буде змога зареєструватись та отримати доступ до приватного Container Registry – репозіторію для Docker Image та, в майбутньому, для Helm Chart.

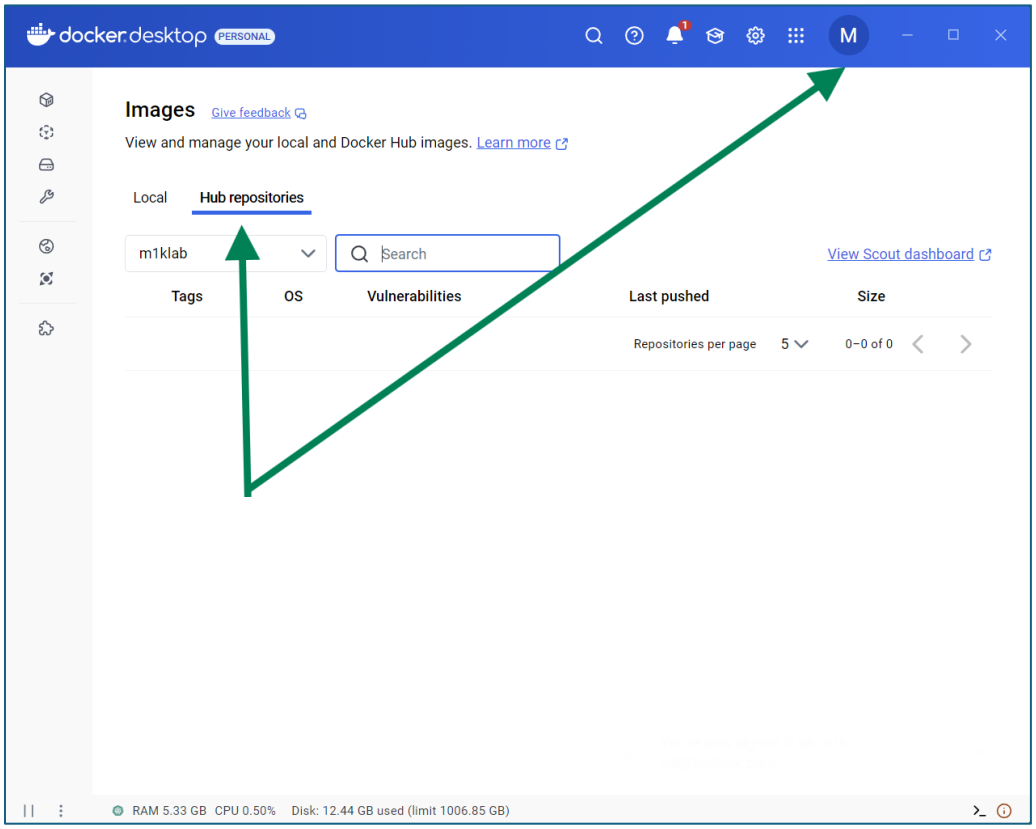
Для цього оберіть Sign In:



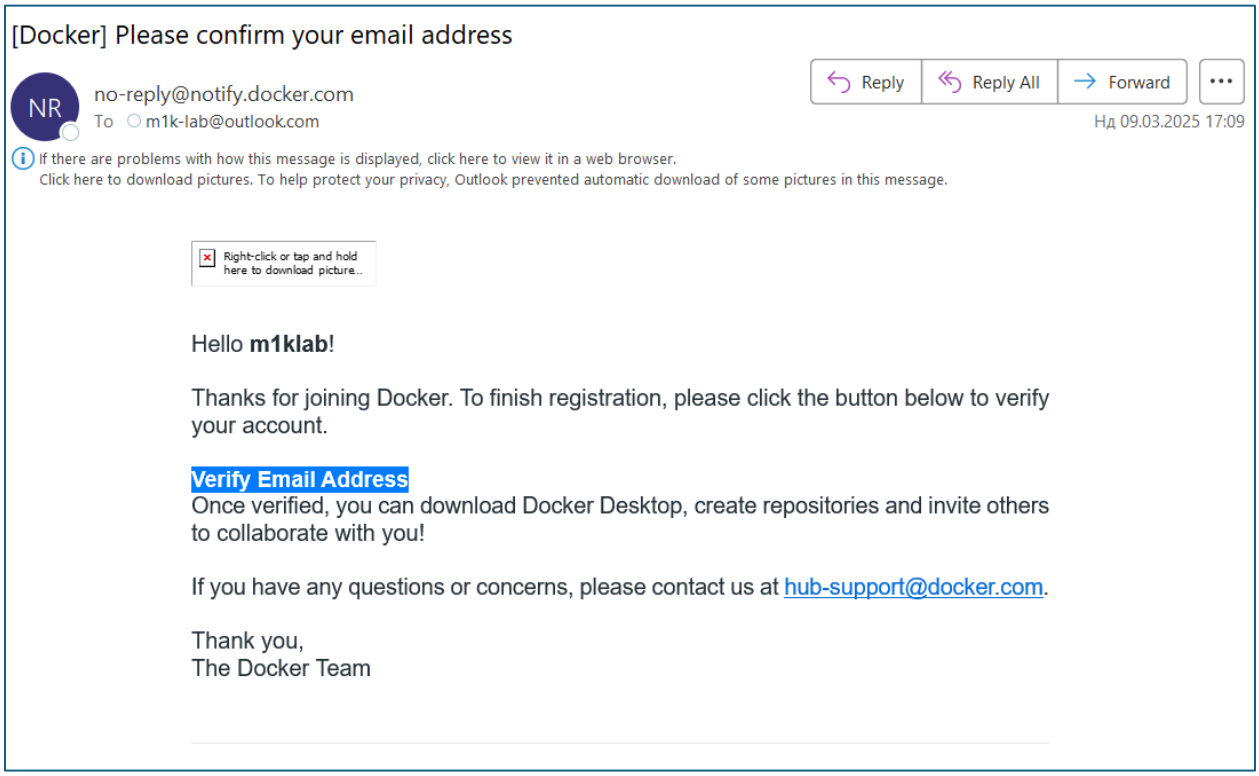
Далі у вас відкроється веб сторінка де ви зможете зареєструватись:



Після реєстрації зайдіть в Docker Desktop та залогінтесь, у вас з’явиться доступ до вкладки Docker Hub, де і будуть розташовуватить ваші артефакти:



Важливо: після реєстрації вам буде надіслан лист з підтвердженням, обов’язково підвердите ваш email адрес.

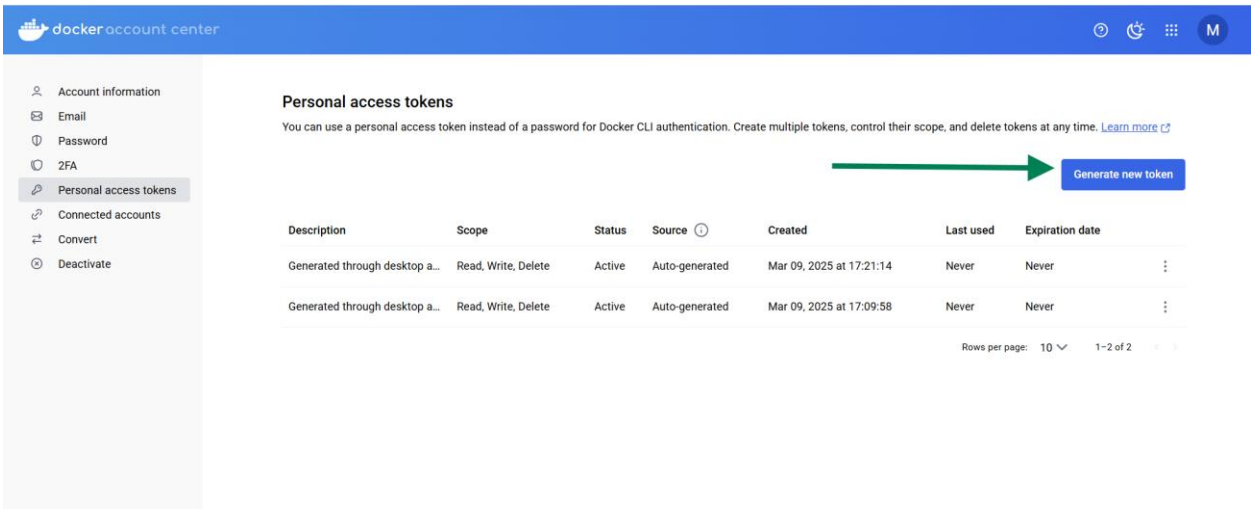


Оскільки у нас безкоштовний план, то розмір сховища лімітован 2Gb, але для наших задач цього буде достатньо.

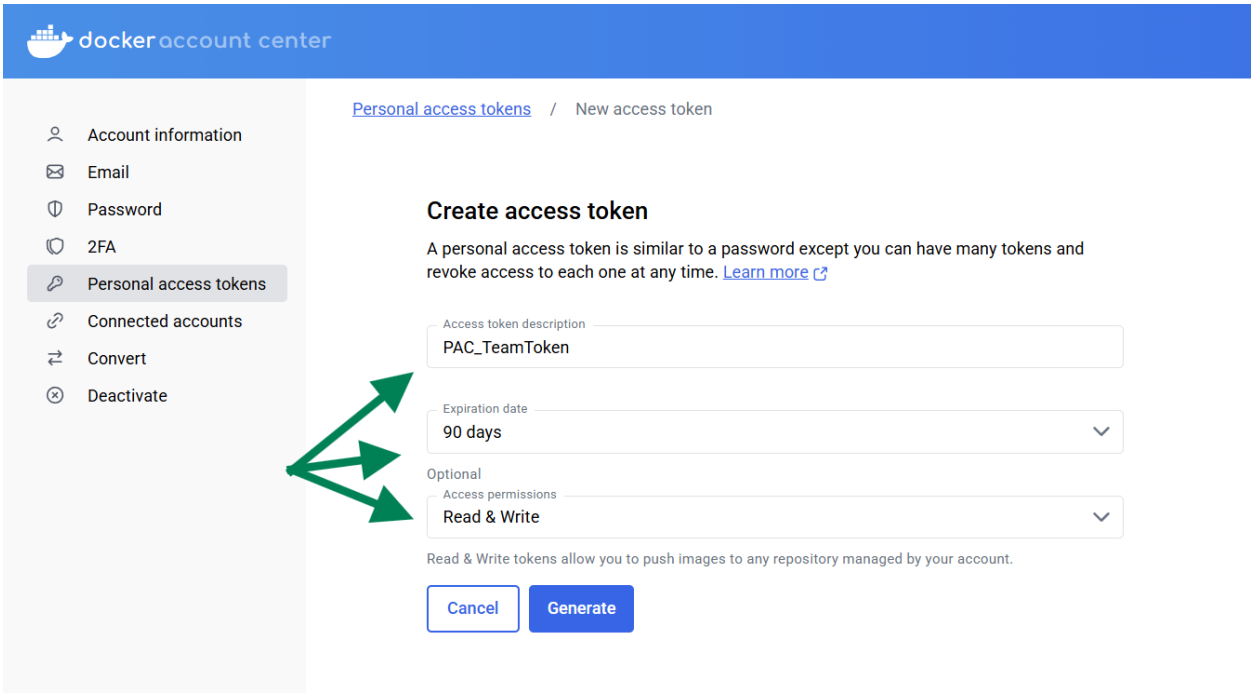
Створення PAT (personal access token)

Для того, щоб кожен член команди мав змогу працювати з Container Registry потрібно сгенерувати token – секретний ключ, що надасть змогу доєднуватись до цього репозіторію.

Для цього зайдіть в свій акаунт -> налаштування -> Personal Access Token

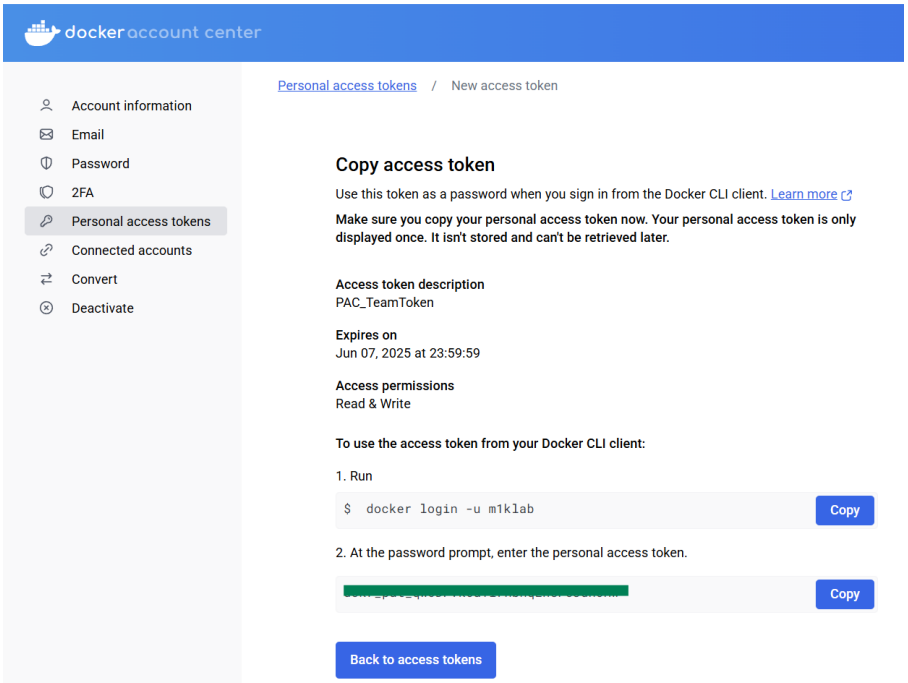


Оберіть сгенерувати новий токен:

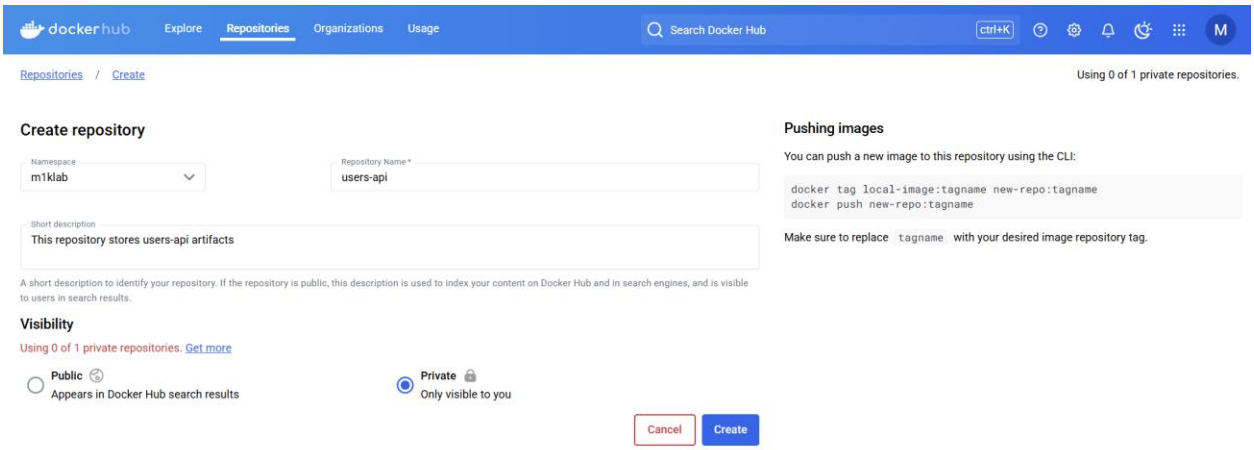


Оберіть назву, термін валідності токену та права на читання та запис.

Обов’язково скопіюйте новостворений токен, тому що після закриття сторінки ви не зможете його знову подивитись.

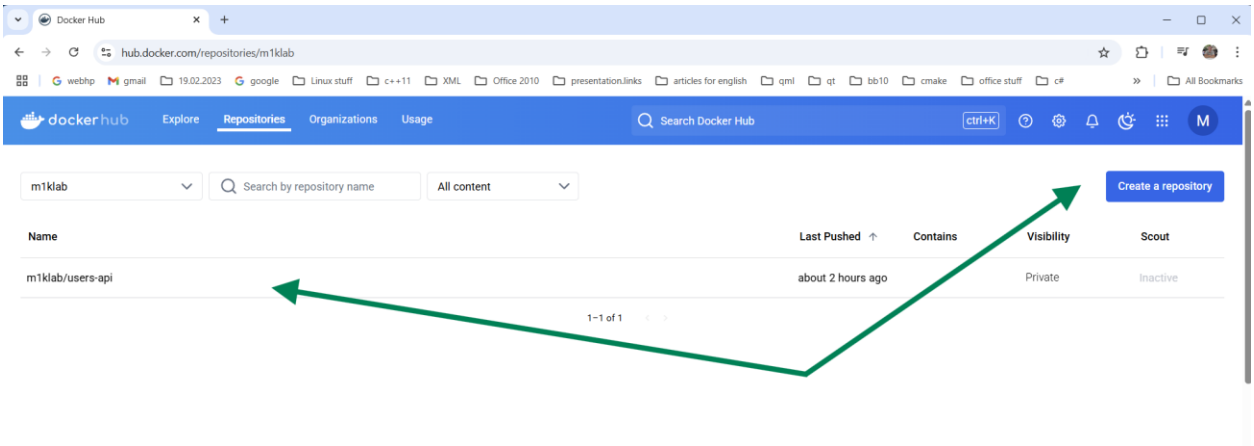


Цей токен потрібно надіслати членам вашої команди, та використовувати як password для доступу к Container Registry з команди docker login.



Створення Container Registry репозіторія

Вам необхідно створити репозіторій в Docker Hub:



При наданні назви репозіторію – обирайте назву вашого сервісу та тип репозіторію – приватний (в безкоштовному плані надається лише один приватний репозіторій). Далі ми будемо «пушити» в цей репозіторій Docker image, що відносяться до вашого сервісу.

Робота з Container Registry

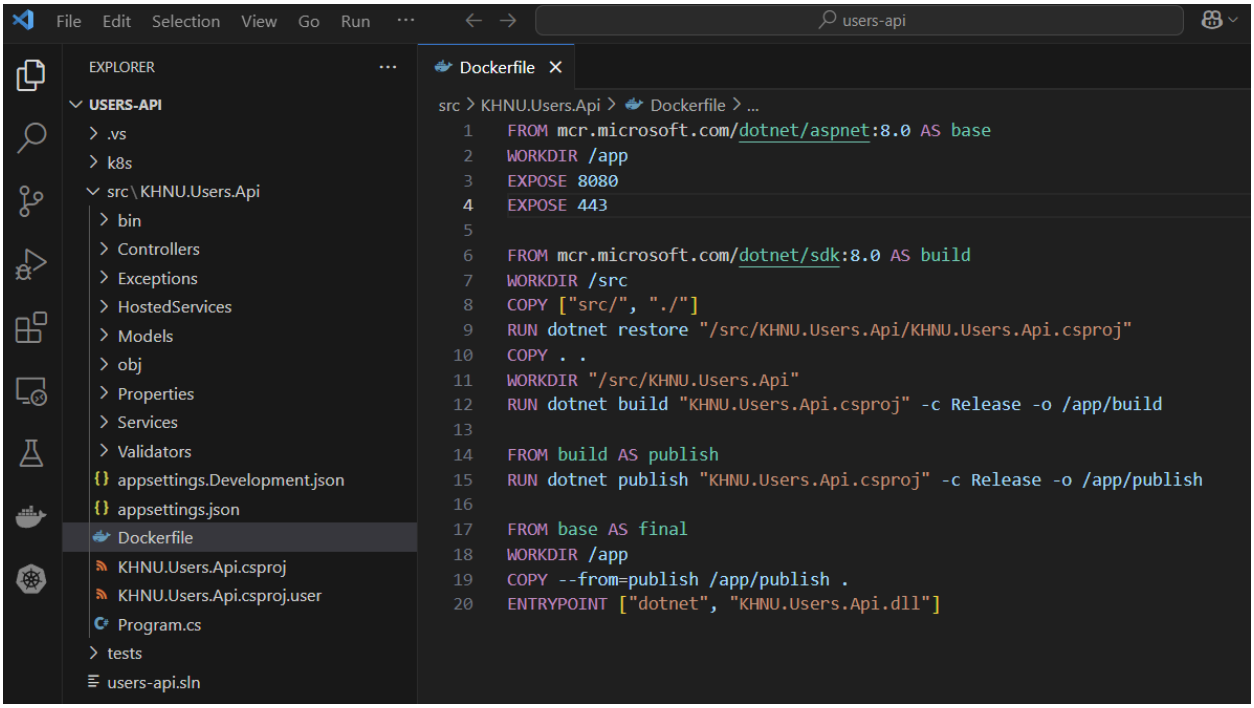
Для того, щоб залити образ в Container Registry нам потрібно буде зробити 4 операції:

1. Залогінетись
   docker login -u <ім’зя користувача в системі Docker Desktop> -p <token що ви сгенерували>

   Приклад: docker login -u m1klab -p 123123123123

2. Зібрати Docker image:
   docker build --platform linux/amd64 -t users-api:0.0.1 -f .\src\KHNU.Users.Api\Dockerfile .

   Це для структури:

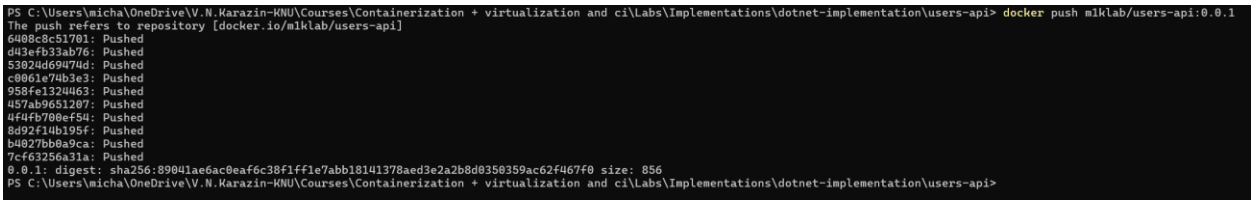


3. Зробити «тегування» новоствореного docker image:
   docker tag <локальна назва образу:тег> <користувач/назва репозіторію: тег>

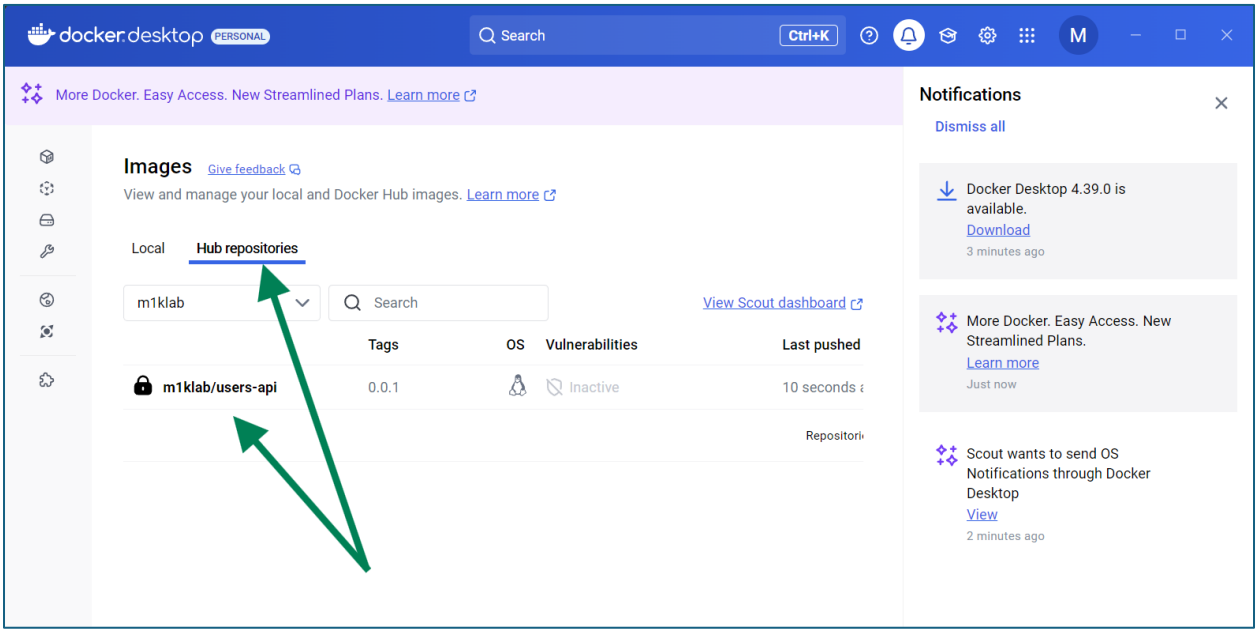
   Приклад:
   docker tag users-api:0.0.1 m1klab/users-api:0.0.1

   Операція «tag» (https://docs.docker.com/reference/cli/docker/image/tag/) це створення посилання на образ, фактично, спосіб перейменування образу. Важливо, щоб ваш образ мав назву, що співвідноситься з репозіторієм, в який ви його будете «пушити», тому операція тегування обов’язкова.

4. Залити образ:
   docker push m1klab/users-api:0.0.1



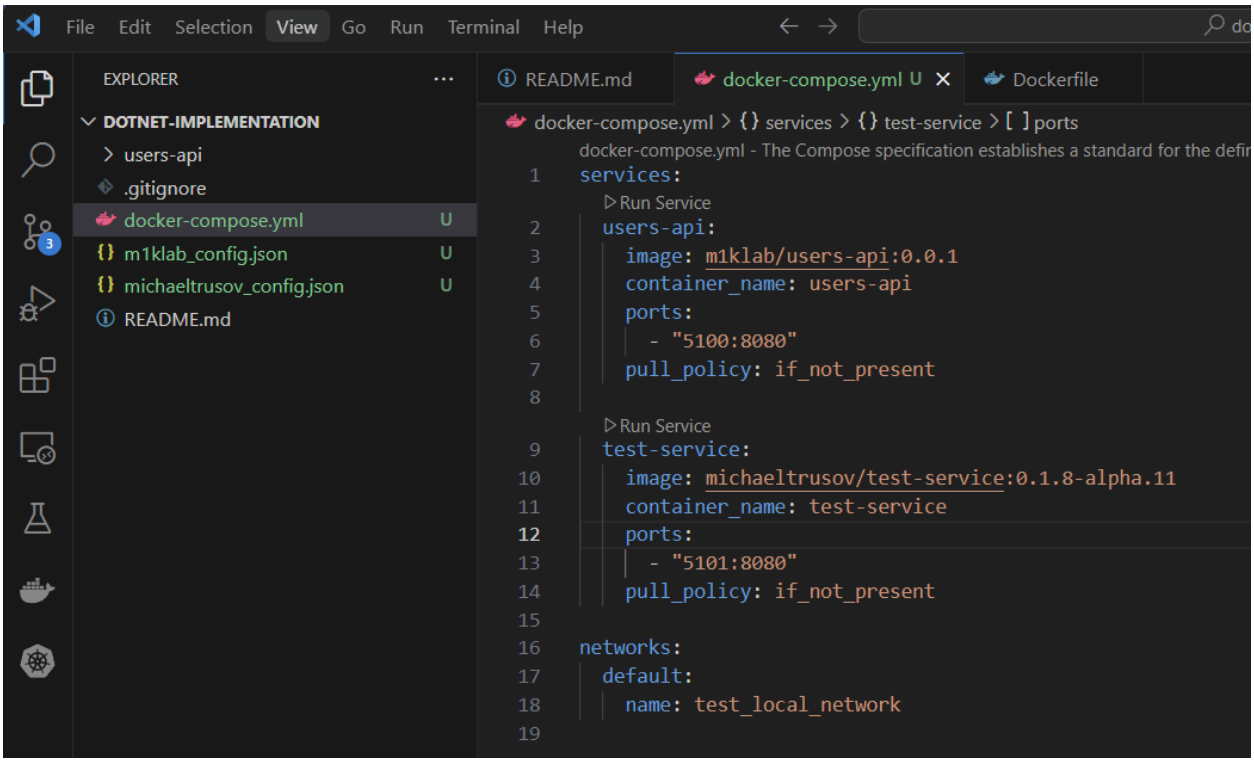
Якщо операція пройшла успішно ви побачете Docker image образ в Docker Hub:



Додаток Д. Утіліта docker-compose

Додаткову інформацію про утіліту docker-compose можна знайти за посиланням: Docker Compose

1. Створимо новий docker-compose.yml документ (приклад):




```
services:
  users-api:
    image: m1klab/users-api:0.0.1
    container_name: users-api
    ports:
      - "5100:8080"
    pull_policy: if_not_present

  test-service:
    image: michaeltrusov/test-service:0.1.8-alpha.11
    container_name: test-service
    ports:
      - "5101:8080"
    pull_policy: if_not_present

networks:
  default:
    name: test_local_network
```

В цьому прикладі ми нмагаємось запустити 2 сервіса: users-api та test-service.

```
…
  users-api:
    image: m1klab/users-api:0.0.1
    container_name: users-api
    ports:
      - "5100:8080"
    pull_policy: if_not_present
…
```

Коментарі:

1) “image:” тут нам потрібно вказати Docker Image, що ми плануєм використовувати, формат: <користувач>/образ:тег
2) “ports:” порти які будть використовуватись для пробросу запитів, формат: <порт локальної машина>:<внутрішній порт контейнеру>
3) “pull_policy:” у нас встановлено занчення – скачувати обрах, якщо його немає на локальній машині.

2. Запуск docker-compose
Оскільки ми лімітовані безкоштовною підпискою і не можемо мати більш одного приватного репозіторія, то сервіси в нашому docker-compose файлу будуть з різних Container Registries, які відносяться до різних користувачів та потребують різну аутентифікацію.
Тому вам потрібно буде спочатку зробити pull docker images, що присутні в docker-compose файлі. Для кожного сервіса, що в приватному репозіторії зробить наступне:
1) docker login -u <користувач>
2) docker-compose pull <назва сервісу в docker-compose файлу>

```
PS C:\Users\micha\OneDrive\V.N.Karazin-KNU\Courses\Containerization + virtualization and ci\Labs\Implementations\dotnet-implementation> docker login -u m1klab
Password:

Login Succeeded
PS C:\Users\micha\OneDrive\V.N.Karazin-KNU\Courses\Containerization + virtualization and ci\Labs\Implementations\dotnet-implementation> docker-compose pull users-api
[+] Pulling 1/1
 ✔users-api Pulled
PS C:\Users\micha\OneDrive\V.N.Karazin-KNU\Courses\Containerization + virtualization and ci\Labs\Implementations\dotnet-implementation> docker login -u michaeltrusov
Password:

Login Succeeded
PS C:\Users\micha\OneDrive\V.N.Karazin-KNU\Courses\Containerization + virtualization and ci\Labs\Implementations\dotnet-implementation> docker-compose pull test-service
[+] Pulling 10/10
 ✔test-service Pulled
   ✔427d07ebfad0 Download complete
   ✔74a1318cd9df Download complete
   ✔dfde04fc2ec3 Download complete
   ✔1d2f51cd947a Download complete
   ✔4f4fb700ef54 Already exists
   ✔88496bad2e81 Download complete
   ✔a870a55f5dde Download complete
   ✔e4fff0779e6d Download complete
   ✔c82d150f73e8 Download complete
```

Далі просто запускайте docker-compose файл:

docker-compose -f .\docker-compose.yml up -d

```
PS C:\Users\micha\OneDrive\V.N.Karazin-KNU\Courses\Containerization + virtualization and ci\Labs\Implementations\dotnet-implementation> docker-compose -f .\docker-compose.yml up -d
[+] Running 3/3
 ✔Network test_local_network  Created                                                                    0.1s
 ✔Container test-service      Started                                                                    0.6s
 ✔Container users-api         Started                                                                    0.6s
```